SE4041 – Mobile Application Design and Development

# Spot Saver

## Assignment 02 – Part A: Advanced iOS App Report

| | |
|---|---|
| **Student name** | Asan M A M |
| **Student ID** | IT22253262 |
| **Module** | SE4041 – Mobile Application Design and Development |
| **Assessment** | Assignment 02, Part A – Advanced iOS App (15%) |
| **Platform** | iOS 26 · SwiftUI · SwiftData · MapKit |
| **Repository** | github.com/SLIIT-IM-Y4S2/it22253262-spot-saver-ios |
| **Date** | 15 October 2026 |

# Contents

1. Purpose and target audience ..... 3
The problem ..... 3
The solution ..... 3
Target audience ..... 3
2. User guide ..... 4
2.1 First launch ..... 4
2.2 Browse the map ..... 5
2.3 Save a new spot ..... 6
2.4 Find spots in the list ..... 7
2.5 Spot details, directions and visits ..... 8
2.6 Dark Mode and landscape ..... 9
3. Technical documentation ..... 11
3.1 Architecture ..... 11
3.2 Project structure ..... 11
3.3 Data persistence ..... 11
3.4 Emerging technology: MapKit and CoreLocation ..... 12
3.5 Custom components and modifiers ..... 12
3.6 Build and run ..... 12
4. Design decisions ..... 13
4.1 Visual design and branding ..... 13
4.2 Animations and gestures ..... 13
4.3 Accessibility and adaptive layout ..... 13
5. Testing and performance ..... 15
5.1 Unit tests (Swift Testing) ..... 15
5.2 UI tests (XCUITest) ..... 15
5.3 User acceptance testing ..... 16
5.4 Performance optimisations ..... 16
6. Challenges and reflections ..... 17
6.1 Development challenges ..... 17
6.2 Reflections ..... 17
6.3 AI-assisted development ..... 17
6.4 References ..... 17

# 1. Purpose and target audience

Spot Saver solves a small but constant problem: people find a good café, a quiet study corner or a free parking lane, then forget exactly where it was or why they liked it.

## The problem

- General map apps show every business, not the places *you* rated, and their saved lists hold no personal notes or visit history.
- Notes apps keep the text but not the location, so getting back to the place takes a second app.
- Photos of a place have a location but no rating, category or comment.

## The solution

Spot Saver puts all of this in one place: a pin on a map with a category, a 0–5 star rating, notes, a photo, the address and a visit history. One tap gives walking or driving directions through Apple Maps.

## Target audience

| Group | Why the app fits them |
|---|---|
| University students (primary) | Constantly look for study spots, cheap food and parking around campus; the sample data is built around SLIIT Malabe |
| People new to a city | Need to remember useful places quickly while still learning the area |
| Commuters and drivers | Save reliable parking and stop-off points, then navigate back with one tap |

The design choices follow from this audience: one-handed use on a phone, a long-press to save a place in under five seconds, large readable text, and full offline storage so saved spots work without data.

# 2. User guide

The app has two tabs, Map and Spots; every task below starts from one of them and takes three taps or fewer.

## 2.1 First launch

On first launch the app asks for location access, with a message explaining why. Choosing **Allow While Using App** enables "Nearest" sorting and centres the map on you. The app still works if you choose **Don't Allow**; the map then opens on SLIIT Malabe.

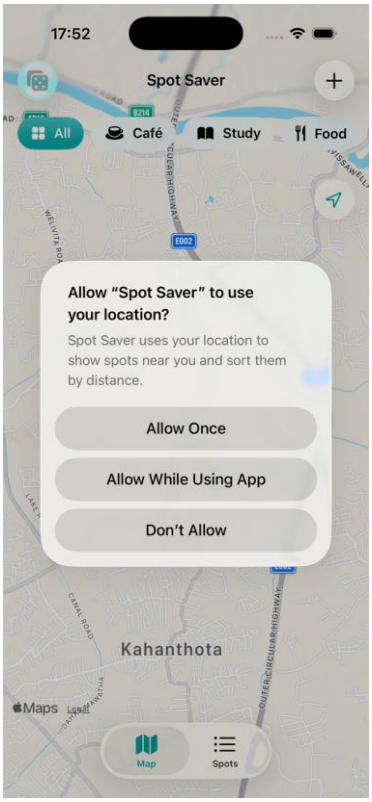



*Figure 1 – Location permission prompt on first launch*

If the list is empty, tap **Spots → Load Sample Spots** to add four demo places around campus.

## 2.2 Browse the map

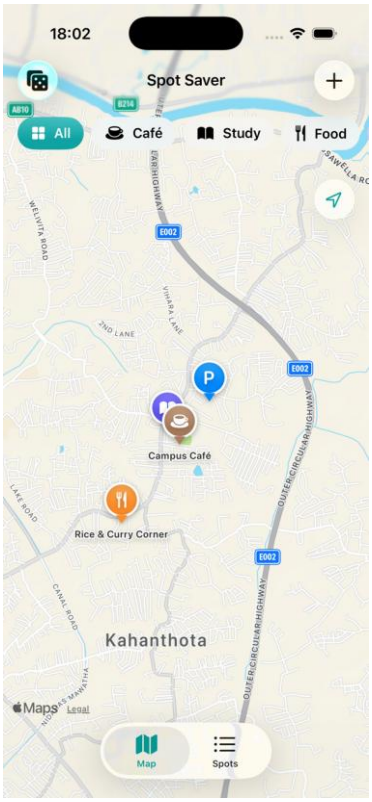



*Figure 2 – Map screen with category chips and pins*

1. Each saved spot is a coloured pin; the colour and icon show its category.
2. Tap a chip (All, Café, Study, Food, Parking, Other) to show only that category.
3. Tap a pin to see a summary card with the name, address and rating. Tap the card to open the full details, or **×** to close it.
4. Tap the dice button (top left) for **Surprise Me**: the map flies to a spot you have not visited for two weeks or more.

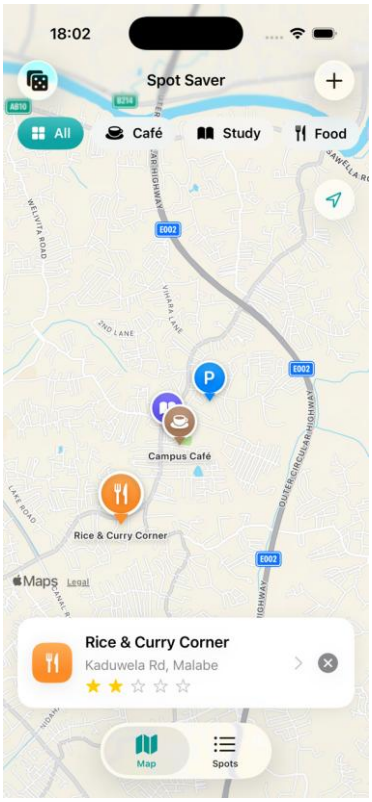



*Figure 3 – A selected pin with its summary card*

## 2.3 Save a new spot

1. **Long-press** anywhere on the map, or tap **+**.
2. Enter a name and choose a category. Tap or drag across the stars to rate it.
3. The address fills in automatically. Tap the small map to move the pin if needed.
4. Optionally add a photo, notes, and mark it as a favourite.
5. Tap **Save**. Save stays disabled until the spot has a name.

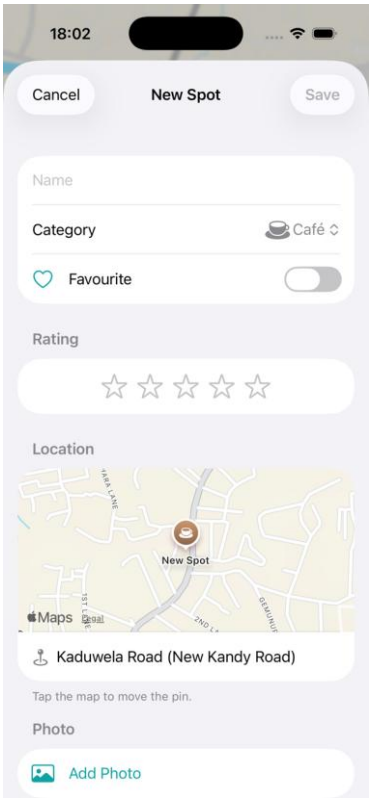



*Figure 4 – New Spot form*

## 2.4 Find spots in the list

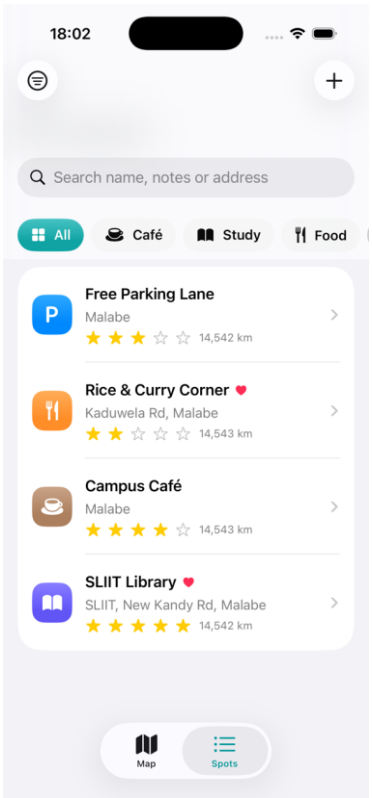



*Figure 5 – Spots list with search and category chips*

- Type in the search bar to match names, notes or addresses.

- Use the filter button (top left) to sort by Newest, Top Rated, Name or Nearest, or to show favourites only.
- Swipe a row left to delete it, or right to mark it as a favourite.

## 2.5 Spot details, directions and visits

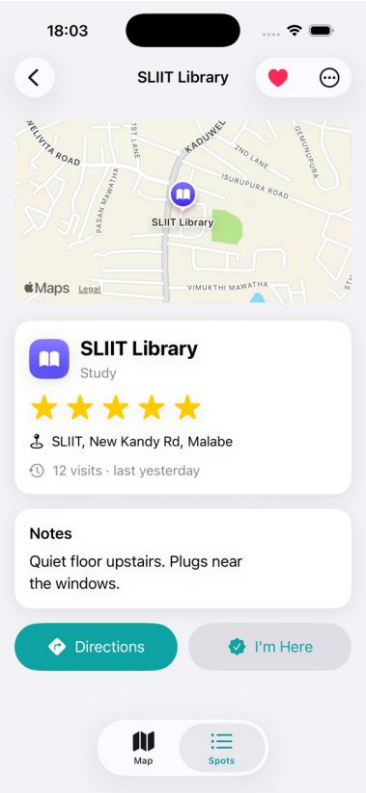



*Figure 6 – Spot detail screen*

- **Directions** opens Apple Maps with a route to the spot.
- **I'm Here** records a visit; the card shows the visit count and when you were last there.
- Tap the heart to favourite it. The **…** menu has **Edit** and **Delete**; delete asks for confirmation.

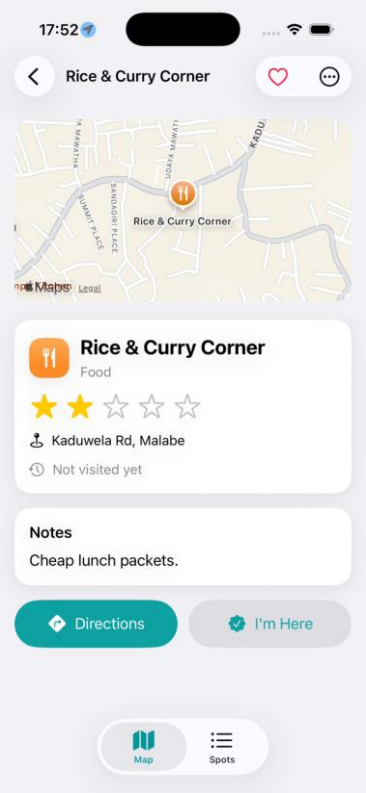



*Figure 7 – Edit and Delete menu*

## 2.6 Dark Mode and landscape

The app follows the system appearance and works in every orientation, and on iPad the tabs become a sidebar.

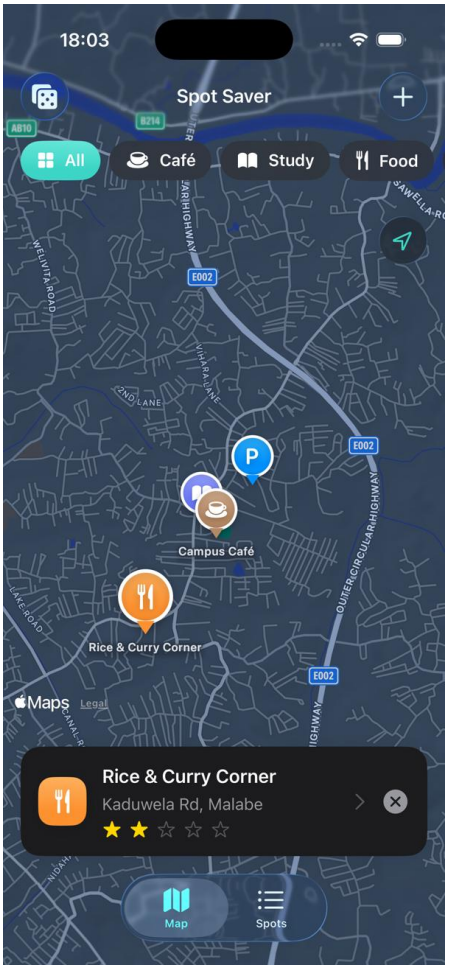



*Figure 8 – Map in Dark Mode*

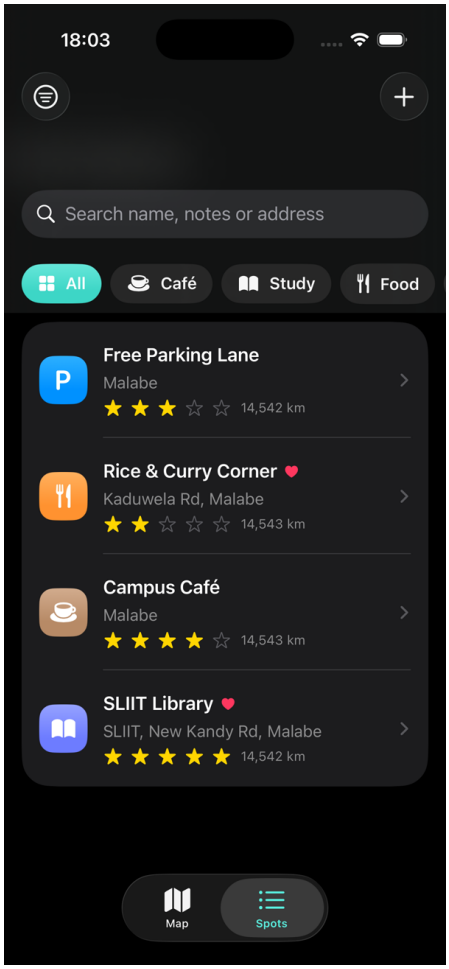



*Figure 9 – Spots list in Dark Mode*

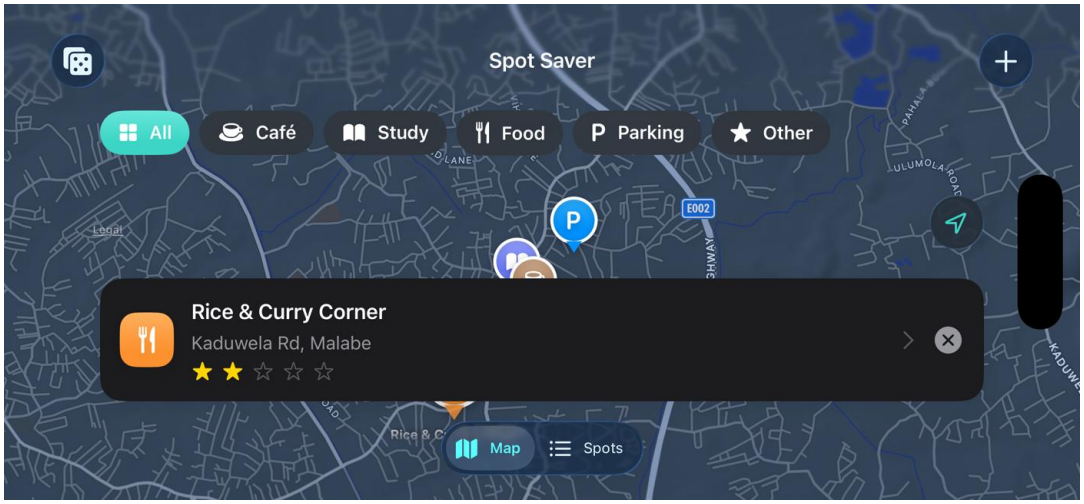



*Figure 10 – Map in landscape*

# 3. Technical documentation

Spot Saver is a SwiftUI app targeting iOS 26.5 on iPhone and iPad, structured in layers so that all search, sorting and validation logic runs without any UI.

## 3.1 Architecture

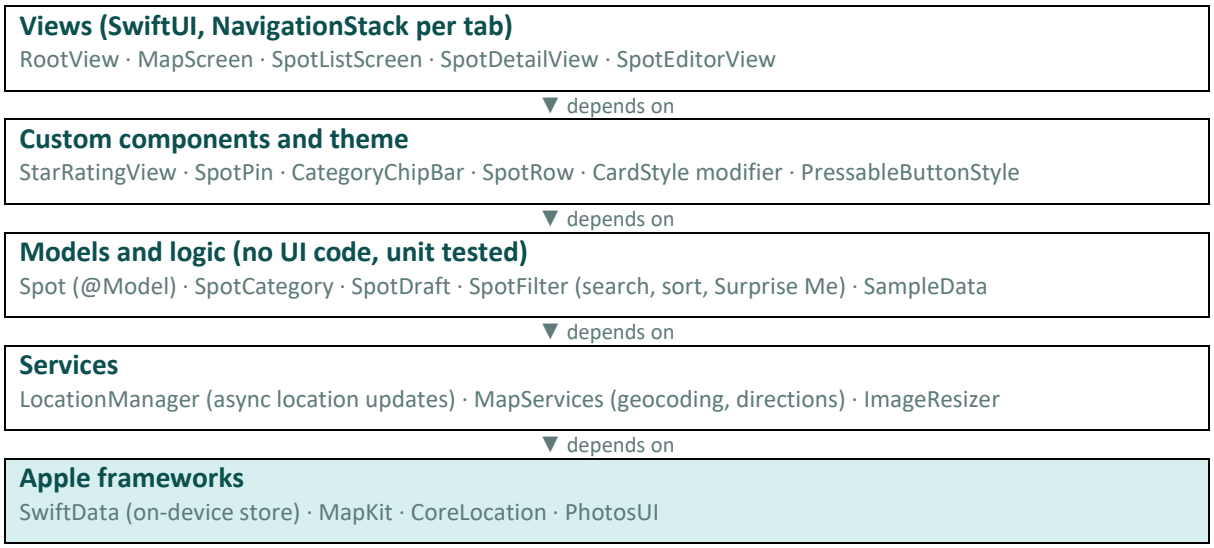



*Figure 11 – Spot Saver architecture: views depend only downward, so the logic is testable without UI*

Views read data through SwiftData's `@Query` and shared state through the SwiftUI environment. Pure logic lives in `SpotFilter` and `SpotDraft`, which is why it can be unit tested without launching the app.

## 3.2 Project structure

| Folder | Contents |
|---|---|
| `SpotSaverApp.swift` | App entry point; creates the SwiftData `ModelContainer` (in-memory during UI tests) |
| `Models/` | `Spot` (@Model), `SpotCategory`, `SpotDraft` (editable copy used by the editor) |
| `Logic/` | `SpotFilter` (search, category, favourites, 4 sort orders, Surprise Me), `SampleData` |
| `Services/` | `LocationManager`, `MapServices` (reverse geocoding, directions, distance), `ImageResizer` |
| `Theme/` | Brand colours, `CardStyle` view modifier, `PressableButtonStyle` |
| `Components/` | `StarRatingView`, `SpotPin`, `CategoryChipBar`, `SpotRow`, `CategoryBadge` |
| `Views/` | `RootView`, `MapScreen`, `SpotListScreen`, `SpotDetailView`, `SpotEditorView` |

## 3.3 Data persistence

Spots are stored on the device with **SwiftData**. The `Spot` model holds the name, notes, category, rating (clamped to 0–5), coordinates, address, favourite flag, created date, last visit and visit count.

- The category is stored as a raw string, so an unknown future value falls back to "Other" instead of crashing.
- Photos use `@Attribute(.externalStorage)`, which keeps large image data outside the main database file and keeps queries fast.
- The editor works on a `SpotDraft` copy; **Cancel** discards it, so half-edited data never reaches the database.

## 3.4 Emerging technology: MapKit and CoreLocation

| Feature | API used |
|---|---|
| Custom animated pins, user location, compass, scale | `Map`, `Annotation`, `UserAnnotation`, `mapControls` |
| Long-press on the map to drop a spot | `MapReader` + `MapProxy.convert(_:from:)` |
| Address filled in automatically | `MKReverseGeocodingRequest` (iOS 26) |
| Directions | `MKMapItem.openInMaps` |
| Live location and "Nearest" sorting | `CLLocationUpdate.liveUpdates()`, `CLServiceSession` |
| Surprise Me camera fly-to | animated `MapCameraPosition` |

The integration is central to the app rather than decorative: a spot *is* a location, and saving, finding and returning to it all run through these frameworks.

## 3.5 Custom components and modifiers

- **StarRatingView** – tap or drag to rate; VoiceOver users swipe up or down to adjust.
- **SpotPin** – a pin that drops in with a spring animation and grows when selected.
- **CategoryChipBar** – filter chips whose highlight slides between them (`matchedGeometryEffect`).
- **CardStyle** – a reusable `ViewModifier` for rounded, elevated cards that adapt to Dark Mode.
- **PressableButtonStyle** – buttons shrink slightly while pressed.

## 3.6 Build and run

1. Open `iOS/asanios/asanios.xcodeproj` in Xcode 26 or later on macOS.
2. Set your signing team under *Signing & Capabilities*.
3. Choose an iPhone simulator and press ⌘R.
4. Optional: *Features* → *Location* → *Custom Location* = 6.9147, 79.9729 (SLIIT Malabe).
5. Run the tests with ⌘U.

# 4. Design decisions

Every design choice aims at one goal: saving or finding a place should take seconds, one-handed, in any lighting.

| Decision | Reason |
|---|---|
| Map as the home screen | Places are remembered by *where* they are; the map gives instant context |
| Long-press to drop a pin | The fastest way to save the spot you are standing at or looking at, without typing coordinates |
| Two tabs, each with its own `NavigationStack` | Map and list keep their own back history; on iPad `.sidebarAdaptable` turns the tabs into a sidebar |
| Colour + icon per category | Pins can be told apart at a glance, even in a dense area |
| Editor works on a draft | **Cancel** is always safe; Save is disabled until a name is entered |
| "Surprise Me" | A small, original feature that brings back spots you have forgotten, instead of only showing favourites |
| Sample data around SLIIT | Lets a first-time user (and the examiner) see a full app immediately |

## 4.1 Visual design and branding

- One brand colour, teal, defined in the asset catalog with separate light and dark values, used for buttons, the selected chip and controls.
- Category colours use system colours (brown, indigo, orange, blue, pink), which adapt automatically to Dark Mode.
- Rounded cards (`CardStyle`) and consistent 16-point spacing on every screen.
- The app name shown on the home screen is "Spot Saver".

## 4.2 Animations and gestures

| Interaction | Implementation |
|---|---|
| Pins drop in and grow when selected | Spring animations on `SpotPin` |
| Highlight slides between filter chips | `matchedGeometryEffect` |
| Summary card slides up from the bottom | `.transition(.move(edge: .bottom))` |
| Camera flies to a spot (Surprise Me) | Animated `MapCameraPosition` |
| Stars bounce when rated | `symbolEffect(.bounce)` + haptic feedback |
| Long-press, drag-to-rate, swipe actions, tap-to-move pin | `LongPressGesture`, `DragGesture`, `swipeActions`, `onTapGesture` |

## 4.3 Accessibility and adaptive layout

- **Dark Mode**: all colours are system or asset colours with dark variants (see section 2.6).
- **Dynamic Type**: only system text styles are used, and custom sizes (pins, stars, badges) scale with `@ScaledMetric`.

- **VoiceOver**: list rows read as one element; the star rating is an adjustable control; decorative images are hidden.
- **Adaptive layout**: the detail screen shows media and information side by side on wider screens, and action buttons stack vertically when space is short (`ViewThatFits`). All orientations are supported.

# 5. Testing and performance

The app has 14 unit tests and 4 UI tests. It built and ran on the iPhone 17 Pro simulator, and every feature shown in section 2 was exercised there by hand.

## 5.1 Unit tests (Swift Testing)

| Area | What is checked |
|---|---|
| Search | Matches name, notes and address, case-insensitively; blank search returns everything |
| Filters | Category and favourites-only filters, alone and combined |
| Sorting | Top Rated (ties broken by newest), Name (natural order: "spot 2" before "Spot 10"), Nearest, and the fallback to Newest without a location |
| Surprise Me | Prefers spots not visited for 14+ days, falls back to all spots, returns nothing for an empty list |
| Model | Rating clamped to 0–5, unknown category falls back to Other, visit counting |
| Persistence | Spots saved into an in-memory SwiftData store are fetched back correctly |
| Draft validation | Blank or over-60-character names rejected; names trimmed; edits applied to the right spot |

## 5.2 UI tests (XCUITest)

| Test | Steps |
|---|---|
| Add a spot | Open Spots → + → check Save is disabled → type a name → Save → spot appears in the list |
| Search | Load sample spots → search "parking" → only Free Parking Lane remains |
| Record a visit | Open a spot → I'm Here → "1 visit" appears |
| Launch performance | Measures app launch time |

UI tests launch the app with `-ui-testing`, which uses an empty in-memory store and skips the location prompt, so every run starts from the same state.

Result of the latest ⌘U run: **to be added** (screenshot of the Test Navigator).

## 5.3 User acceptance testing

Testers were asked to complete five tasks without help: save a spot by long-pressing, find a spot by search, filter by category, get directions, and record a visit.

| Tester | Tasks completed (of 5) | Rating (of 5) | Feedback |
|---|---|---|---|
| Ashen K | 4 | 5 | I love how the icons are showing in the map its like apple my with my customization |
| Muthula | 5 | 4 | I like the menu bar it easy to navigate, and it get me where I want |
| Janith | 4 | 5 | its not overcomplicating it dose what it has to do so that's good |

## 5.4 Performance optimisations

- **Memory**: photos are downscaled to at most 1,024 px and saved as JPEG (a full camera photo can be over 10 MB), and stored outside the database with external storage.
- **Responsiveness**: location updates smaller than 25 m are ignored, which avoids needless redraws and saves battery.
- **Load time**: the address lookup runs asynchronously and is cancelled automatically when the pin moves again; the UI never waits for it.
- **Efficient lists**: filtering and sorting are done once per render in `SpotFilter`; list rows show a category badge instead of decoding photos.

# 6. Challenges and reflections

The biggest challenge was not technical: the app was developed on Windows, and Xcode runs only on macOS.

## 6.1 Development challenges

| Challenge | How it was handled |
|---|---|
| No Mac available | Code was written on Windows, and the project was built, run and screenshotted on a friend's Mac. The app compiled and ran on the first build |
| Testing location features in a simulator | A fixed custom location (SLIIT Malabe) and built-in sample data make results repeatable |
| Long-press on a map that also pans and zooms | A sequenced long-press then drag gesture, attached with `simultaneousGesture` so normal map gestures keep working |
| Large photos slowing the app | Photos are downscaled before saving and kept in external storage |
| Cancel leaving half-saved edits | The editor edits a `SpotDraft` copy and only writes to SwiftData on Save |
| Distance shown in thousands of km in early screenshots | The simulator was still set to its default US location; setting the custom location fixes it |

## 6.2 Reflections

- Separating logic (`SpotFilter`, `SpotDraft`) from views made the tests short and fast, and would make it easy to add an iCloud sync layer later.
- Building around Apple's newest APIs (SwiftData, `CLLocationUpdate`, `MKReverseGeocodingRequest`) removed a lot of boilerplate compared with older delegate-based code.
- Next steps: iCloud sync between devices, sharing a spot with friends, a home-screen widget for nearby spots, and a custom app icon.

## 6.3 AI-assisted development

As required by the brief, AI use is declared. Claude Code (Anthropic) was used to summarise the brief, suggest app ideas, generate the initial project structure, code and tests, and draft this report. All output was reviewed and built in Xcode. The prompts and responses are recorded in the `AI Usage Log` folder of the submission.

## 6.4 References


- Apple Developer Documentation: SwiftUI, SwiftData, MapKit for SwiftUI, Core Location
- Apple Human Interface Guidelines